Computación

Clase 8

Temas

- Procedimientos
- Ámbito de las variables
- Pasaje de parámetros
- Anidamiento de subprogramas

Definición de procedimiento

Un procedimiento es un subprograma que puede recibir cero, uno o más parámetros y devolver al módulo que lo invoca: ninguno, uno o más valores.

Declaración de un procedimiento

Ident. del procedimiento

```
PROCEDURE  PROCE1 <(pi,p2...pn)>;
........
Begin
......
........

End;
```

Cuerpo

Lista de parámetros formales (opcional)

Llamado a un procedimiento

```
PROGRAM principal;
 CONST
 TYPE
  VAR
   A,B,C:integer;
   PROCEDURE Procedim1 (A,B,C:integer);
    .............
    Begin
    .......
    ......
    end;

   Begin
   {Sentencias ejecutables del programa}
   Procedim1(A,B,C);
   .......
   End.
```

1

2

Ejemplo de declaración de un procedimiento

```
PROCEDURE Nombre(param1:char;cant:integer);
 Var
  j:integer; {variable local}
BEGIN
  For j:=1 to cant do
        Write(param1);
   Writeln
END;
```

Ejemplo de llamado de un procedimiento

```
Var
W:char;
Z:byte;
BEGIN {seccion ejecutable}
  . . . . . .
  Nombre('@',5);
  . . . . . . .
  Nombre(w,z);
  . . . .

End.
```

Procedimientos: ejemplo

Dibujar, N veces, el siguiente esquema:

```
+ + + + + + + + + +
* * * * * * * * * * * * * * * * * * * *
```

Procedimientos: ejemplo

```
PROGRAM trama2;
Var
i,veces:integer;
caract1,caract2:char;
cant1,cant2:integer; {variables globales}

{Definición del procedimiento con parámetros}

PROCEDURE dibujo(caracter:char;cant:integer);
Var
  j:integer; {variable local}
BEGIN
    For j:=1 to cant do
          Write(caracter);
     Writeln
END;
```

Parámetros formales

Procedimientos: ejemplo

```
BEGIN   {p.p.}
   Writeln('Ingrese las veces que repite  los dibujos');
   Readln(veces);
   Write('Cant. de caracteres para el primer dibujo');
   Readln(cant1);
   Write('carácter del primer dibujo');
   Readln(caract1);
   Write('Cant. de caracteres para el segundo dibujo');
   Readln(cant2);
   Write('carácter del segundo dibujo');
   Readln(caract2);
   For i:=1 to veces do
         Begin
              dibujo(caract1,cant1);
              dibujo(caract2,cant2)
         End;
 END.
```

El procedimiento se lo invoca a través de su nombre y los parámetros, creando una nueva sentencia

Ámbito de las variables

```
PROGRAM identificador;
CONST
Definición de constantes
TYPE
Declaración de tipos
VAR
Declaracion de variables de alcance global

  PROCEDURE  PROCE1 (parámetros formales);
  var
  Declaración de variables locales
  Begin
  ......
  ........
  End;
BEGIN
PROCE1(parámetros actuales);
. . . . . .
END.
```

Ámbito de las variables: ejemplo

```
PROGRAM identificador;
CONST
Definición de constantes
TYPE
Declaración de tipos
VAR
 Glob1,glob2:real;

 PROCEDURE  PROCE1 (parámetros formales);
 var
 Local1,local2:integer;
 Begin
 ......
 ........
 End;
BEGIN
Proce1(parámetros actuales);
. . . . . .
END.
```

Aclaración

Si bien la implementación del lenguaje permite modificar las variables de alcance global en cualquier punto del programa, se considera una ***mala práctica de programación*** *extraer resultados de los procedimientos a través de ellas. Recordar que el mecanismo de comunicación entre el programa principal (u otros procedimientos) y los procedimientos debe hacerse SÓLO a través de los parámetros.*

Pasaje de parámetros

Comunicación entre el programa principal u otros subprogramas y los procedimientos

Pasaje de parámetros por valor

Pasaje de parámetro por valor: ejemplo

```
PROGRAM Ejemplovalor;
Var
    x:integer;
PROCEDURE Cambio(y:integer);
Begin
          y:=1
End;
BEGIN
     x:=0;
     Cambio(x);
     Writeln('El valor de X es', x)
END.
```

Parámetro formal por valor

Parámetro actual por valor

Pasaje de parámetro por valor: ejemplo

```
PROGRAM Ejemplovalor;
Var
    x:integer;
PROCEDURE Cambio(y:integer);
Begin
            y:=1
End;
BEGIN
   x:=0;
   Cambio(x);
   Writeln('El valor de X es', x)
END.
```

Se establece una comunicación unidireccional entre los parámetros a fin de establecer el pasaje de información

Pasaje de parámetro por valor

- ✓ Al parámetro formal se le asigna memoria y se le copia el valor del parámetro actual y se corta la comunicación entre el programa principal y el procedimiento.

- ✓ Si se modifica el valor del parámetro formal dentro del procedimiento, el valor del parámetro actual NO CAMBIA.

- ✓ Si bien el parámetro formal debe ser una variable, el parámetro actual puede ser una expresión.

Pasaje de parámetros por valor : sintaxis

```
Procedure Ejemplo(a,b:integer;zeta:real);
```

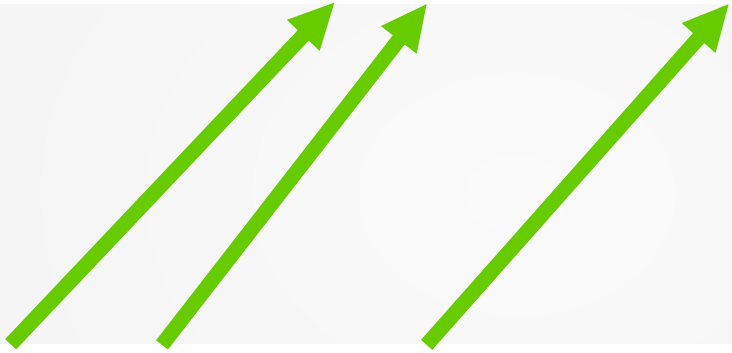
Los parámetros formales por valor se declaran en una lista similar a la declaración de variables de la sección VAR

```
Ejemplo(alt,2*beta,z);
```

Los parámetros actuales por valor se denotan usando expresiones del mismo tipo que su correspondiente parámetro formal

Pasaje de parámetro por valor: ejemplos

PROCEDURE Ejem(A , B:integer; C:real); {declaración}



Ejem((X+2)*Z, 7 , Cos(t)*sqrt(Z)); {sentencia de llamado}

Pasaje de parámetros por dirección, variable o referencia

Pasaje de parámetros por variable: ejemplo

```
PROGRAM Ejemplovariable;
Var
    x:integer;
PROCEDURE Cambio(var y:integer);
Begin
     y:=1
End;
BEGIN
     x:=0;
     Cambio(x);
     Writeln('El valor de X es', x)
END.
```

Parámetro formal por variable

Parámetro actual por variable

Pasaje de parámetros por variable: ejemplo

```
PROGRAM Ejemplovariable;
Var
    x:integer;
PROCEDURE Cambio(var y:integer);
Begin
            y:=1
End;
BEGIN
     x:=0;
     Cambio(x);
     Writeln('El valor de X es', x)
END.
```

Se establece una comunicación bidireccional entre los parámetros a fin de generar el pasaje de información

Pasaje de parámetros por variable : sintaxis

```
Procedure Ejemplo(var a,b:integer;var zeta:real);
```

Los parámetros formales por variable se declaran en una lista similar a la declaración de variables de la sección VAR, pero debe escribirse la palabra **var** antecediendo cada declaración

```
Ejemplo(x, b,w);
```

Los parámetros actuales por variable se denotan usando variables (solamente) del mismo tipo que su correspondiente parámetro formal. El nombre puede coincidir o no.

Pasaje de parámetros por variable

- ✓ Cada vez que se modifica un parámetro formal, se modifica el parámetro actual correspondiente. De esta forma, se pueden intercambiar datos. El programa principal le envía datos al procedimiento y el procedimiento le envía resultados al programa principal.

- ✓ No hay copia de valores. Se establece una referencia entre ellos, es decir, el parámetro actual le pasa la dirección donde él se encuentra, al parámetro formal.

Ejemplo completo

```
Program Completo;
 Var dato1:integer; dato2,dato3:real;
Procedure ejem1(param1:integer; VAR param2,param3:real);
 Var V1,V2:integer;
 Const
  C1=1.5;C2=2.5;
Begin
 Writeln('Ingrese un dato entero');Readln(V1);
 V2:=V1*param1;
 param2:=param2/C1;
 param3:=V2*C2;
End;
Begin {P.P.}
 Writeln('Ingrese un dato entero y uno real');
 Readln(dato1,dato2);
 ejem1(2*dato1,dato2,dato3);
 Writeln('El resultado es:',dato1,dato3:4:2,'y',dato2:4:2);
 Readln
End.
```

Anidamiento de procedimientos

- ✓ Un procedimiento que está anidado dentro de otro es privado de ese procedimiento.
- ✓ El mantenimiento de procedimientos privados puede disminuir los errores de programación al limitar el uso de procedimientos a secciones específicas del programa

```
Program Ejemplo:
    procedure Externo1:
        begin
        end;
    procedure Externo2;
        procedure Interno1;
            begin
            end;
        begin
        end;
    procedure Externo3;
        procedure Interno2;
            begin
            end;
        begin
        end;
  begin
  end.
```

NIVEL 1

NIVEL 2

NIVEL 2

NIVEL 2

NIVEL 3

NIVEL 3

```
program anidamiento;
type
   Tcadena = string[15];
var {variables globales}
   usuario,clave:Tcadena;
PROCEDURE cargar_usuario(var usuario:Tcadena;var clave:Tcadena);

   PROCEDURE generar_clave(usuario:Tcadena;var clave:Tcadena);
   var {variables locales a generar_clave}
      i,offset:byte;
   begin {sección ejecutable de generar_clave}
     randomize;
     clave:='';
      for i:=1 to length(usuario) do
       begin
         offset:=random(32)-16;
         clave:=clave+chr(ord(usuario[i])+offset)
       end
   end;
begin {sección ejecutable de cargar_usuario}
  writeln('Ingrese nombre de usuario');
  readln(usuario);
  generar_clave(usuario,clave)
end;
BEGIN {programa principal}
cargar_usuario(usuario,clave);
writeln('La clave para el usuario ',usuario,' es: ',clave)
END.
```